PHP Fundamentals

Conditions, Loops, Arrays & Forms

A practical guide to PHP's core building blocks — from logic and loops to arrays and form handling. Built for learners who want to understand *why* things work, not just copy the code.

PHP BASICS

BEGINNER-INTERMEDIATE

Use **if...elseif...else** to check **multiple conditions** in order. PHP tests each condition from top to bottom and runs the first one that is `true`. The `else` block is the fallback.

How It Works

- `if` - checks the first condition
- `elseif` - checks another condition if the first is false
- `else` - runs when no condition matches

💻 Example

```php
<?php
$marks = 75;
if ($marks >= 80) {
    echo "Excellent";
} elseif ($marks >= 50)
{
    echo "Pass";
} else {
    echo "Fail";
}
?>
```

With `$marks = 75`, the output is **Pass** because the first `elseif` condition is true.

2.4 Switch Statement

Use the `switch` statement when one variable needs to be matched against **multiple fixed values**. It is cleaner than a long chain of `elseif` blocks.

Example

```php
<?php
$day = "Monday";
switch ($day) {
    case "Monday":
        echo "Start of the work week";
        break;
    case "Friday":
        echo "End of the work week";
        break;
    case "Saturday":
    case "Sunday":
        echo "Weekend";
        break;
    default:
        echo "Midweek day";
}
?>
```

Key Points

- `case` checks the variable against a fixed value
- `break` ends the match - without it, PHP falls through to the next case
- `default` runs when no case matches, like `else`

> Use `break` unless you want fall-through behavior.

PART 3

Loops (Iteration)

Loops let you **run the same block of code multiple times** without repeating it. They're useful for everything from printing a list to processing database records.

FOR

Known number of iterations

WHILE

Runs while a condition is true

DO...WHILE

Runs once, then checks

FOREACH

Used for arrays

3.1 FOR Loop

The `for` loop is ideal when you **know exactly how many times** a block of code should repeat. It combines setup, testing, and updating into one compact line.

Syntax

```
for (init; condition; increment) {
    // code to repeat
}
```

How It Reads

- **Init** - set the starting value (`$i = 1`)
- **Condition** - keep going while true (`$i <= 5`)
- **Increment** - update after each run (`$i++`)

Example

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i . "<br>";
}
?>
```

This prints 1 through 5, each on a new line. The loop starts at `$i = 1`, runs while `$i` is 5 or less, and increases by 1 each time.

2.3 IF...ELSEIF...ELSE

3.2 WHILE Loop

The `while` loop keeps running a block of code **as long as its condition is TRUE**. Unlike the `for` loop, you update the counter separately, which is useful when you do not know the number of iterations in advance.

Example

```php
<?php
$i = 1;
while ($i <= 5) {
    echo $i . "<br>";
    $i++;
}
?>
```

How It Works

- `$i = 1` - set the counter before the loop
- `$i <= 5` - check the condition before each pass
- `$i++` - increase the counter inside the loop

> ⚠️ If you do not increment `$i`, the loop never ends - this is an **infinite loop**.

3.3 DO...WHILE Loop

The `do...while` loop runs the code block first, then checks the condition. That means it always runs at least once, even if the condition starts false — useful for menus, prompts, and validation.

Key Distinction

- **while** — checks the condition before running
- **do...while** — checks the condition after running

Even if `$i = 10` and the condition is `$i <= 5`, a `do...while` loop still runs once.

Example

```php
<?php
$i = 1;
do {
    echo $i . "<br>";
    $i++;
} while ($i <= 5);
?>
```

Output: `1 2 3 4 5`. The loop body runs first, then PHP checks whether to continue.

3.4 FOREACH Loop

The `foreach` loop is built for arrays. PHP moves through each item and assigns it to a temporary variable, so you can use the value without managing a counter.

Example

```php
<?php
$names = ["Kelvin", "John", "Mary"];
foreach ($names as $name) {
    echo $name . "<br>";
}
?>
```

It prints each name on a new line: **Kelvin**, **John**, **Mary**.

Why Use FOREACH?

- No manual index tracking
- Works with indexed and associative arrays
- Stops after the last element
- Best choice for array loops

PART 4

Arrays in PHP

An array stores multiple values in one variable. Instead of creating `$name1`, `$name2`, and `$name3`, you keep them together in one `$names` array. PHP supports three array types, each for a different use case.

Indexed Array

Values use numeric indexes starting at 0

Associative Array

Values use named keys as key-value pairs

Multidimensional Array

Arrays inside arrays for complex data

4.1 Indexed Array

An **indexed array** stores values in a numbered list starting at `0`. Each item has a numeric index you use to access it.

How Indexing Works

- `$colors[0]` -> **"Red"**
- `$colors[1]` -> **"Blue"**
- `$colors[2]` -> **"Green"**

PHP arrays are **zero-indexed**, so the first element starts at `0`.

Example

```
$colors = ["Red", "Blue", "Green"];
echo $colors[0];
echo $colors[1];
echo $colors[2];
```

Use square brackets and the index number to access an array element.

4.2 Associative Array

An associative array uses named keys instead of numeric indexes. It works well for storing related data about one item, such as a student record.

Example

```php
$student=[
  "name"=>"Kelvin",
  "age"    => 25,
  "course" => "IT"
];
echo $student["name"];   // Kelvin
echo $student["age"];    // 25
echo $student["course"]; // IT
?>
```

Key Concepts

- The => operator assigns a value to a key
- Access values with the key in quotes: $student["name"]
- Use clear, consistent key names
- Great for real-world data like users, products, and orders

4.3 Multidimensional Array

A **multidimensional array** is an array that contains other arrays. Think of it like a spreadsheet with rows and columns. You access values with **two sets of brackets**: one for the outer array and one for the inner array.

Accessing Elements

- `$students[0][0]` → **"Kelvin"** (row 0, column 0)
- `$students[0][1]` → **25** (row 0, column 1)
- `$students[1][0]` → **"Mary"** (row 1, column 0)

Example

```php
<?php
$students = [
    ["Kelvin", 25],
    ["Mary", 22],
    ["John", 23]
];
echo $students[0][0]; // Kelvin
echo $students[0][1]; // 25
echo $students[1][0]; // Mary
?>
```

Each inner array stores one student's name and age. Add more columns by extending the inner arrays.

Part 5: Arrays with Loops

Arrays and loops are a common PHP pattern. A `foreach` loop moves through each array element automatically, so you do not need to track indexes. Use it to display product lists, print reports, or process form submissions.

Example

```php
<?php
$numbers = [10, 20, 30, 40];
foreach ($numbers as $num) {
    echo $num . "<br>";
}
?>
```

Output: `10`, `20`, `30`, `40` — each on its own line.

Topic 3 Preview

Forms & Data Handling in PHP

Next, we'll look at how PHP collects user input from HTML forms. This powers login pages, search boxes, and registration systems.

LEARNING OUTCOME 2.1.3

Use HTML forms and links to pass and handle data, including user input and file downloads.

PART 6

Functions in PHP

Functions are self-contained blocks of code designed to perform a specific task. They allow you to organize your code, make it reusable, and improve its readability, saving time and reducing errors.

Code Reusability

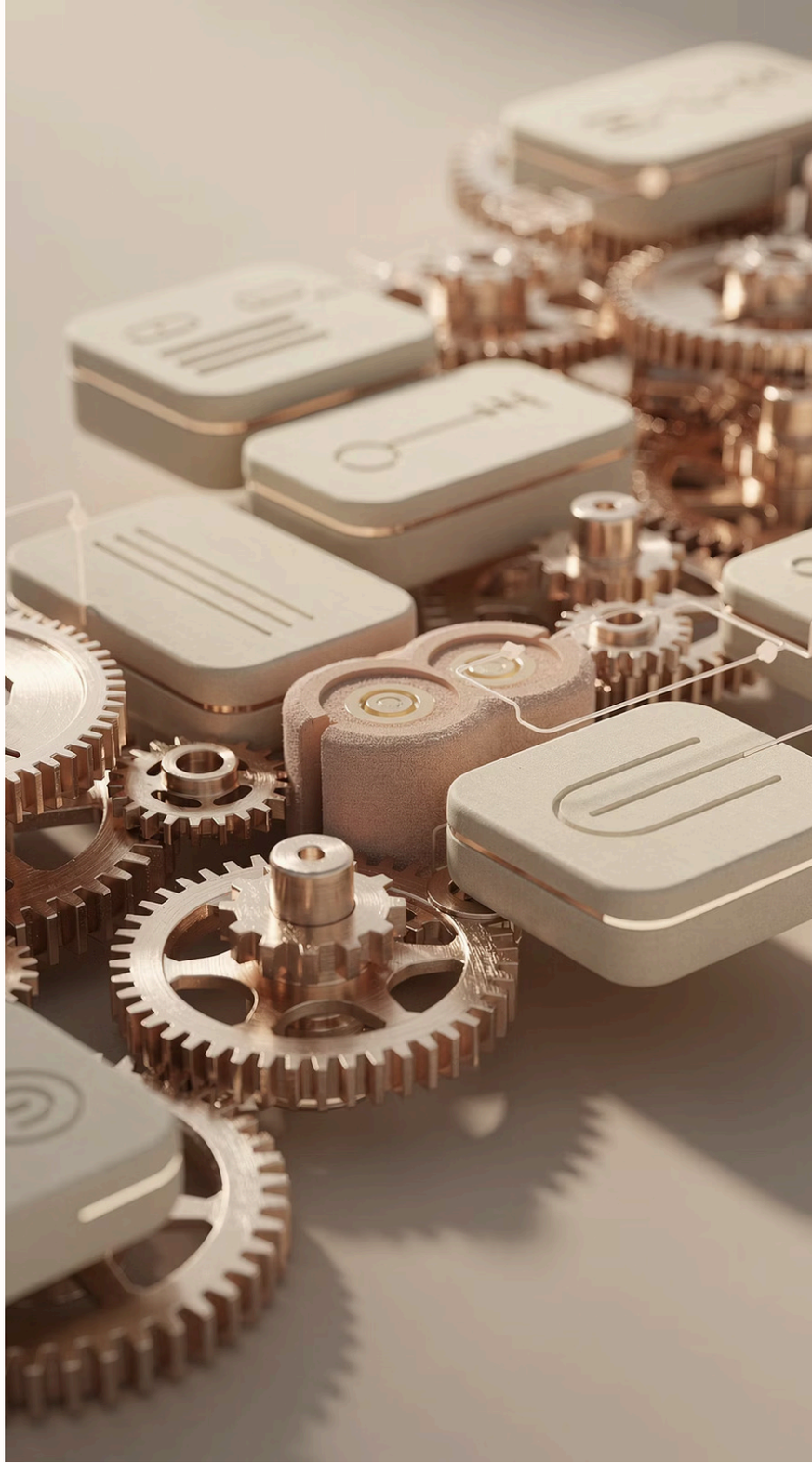
Write once, use many times.

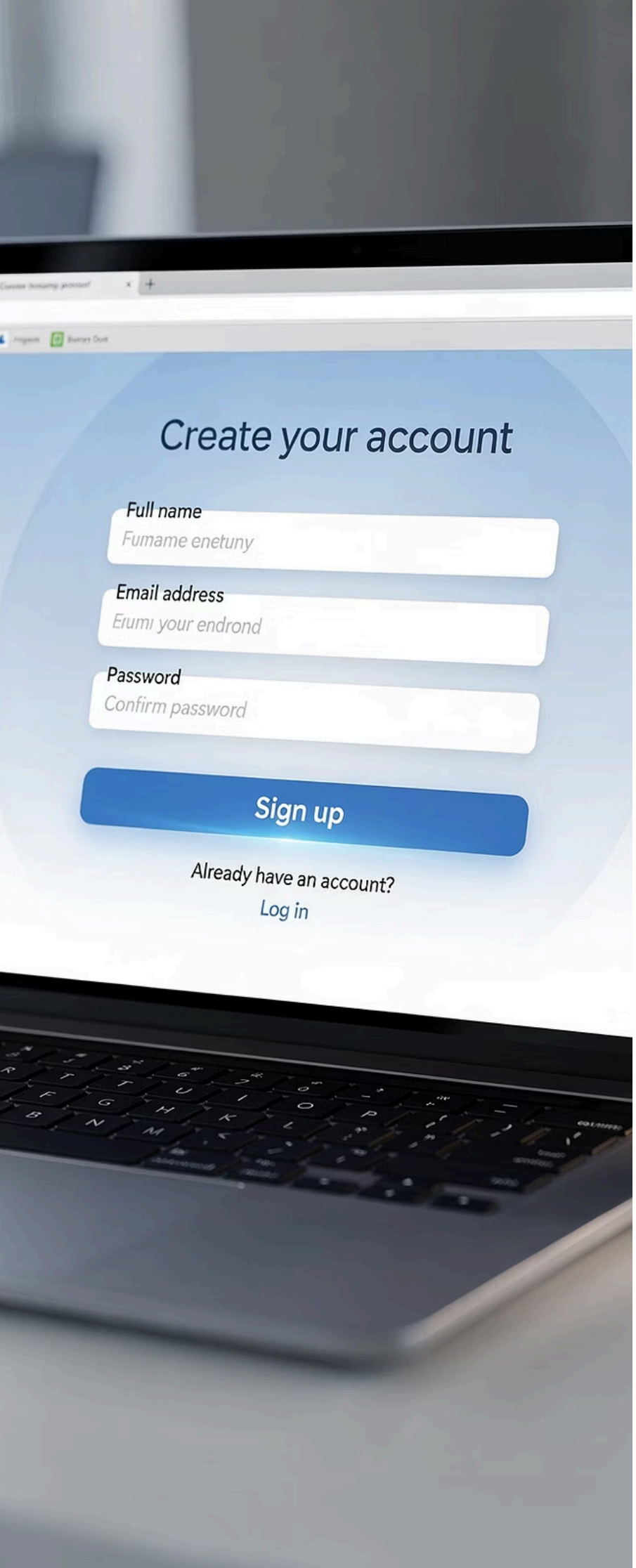
Modularity

Break complex problems into smaller, manageable parts.

Maintainability

Easier to debug and update.





TOPIC 3 · PART 1

Introduction to HTML Forms

A **form** is the main way users send input to a web application. It collects data and sends it to the server for processing. Most web apps use forms for tasks like logging in or searching.

Login Forms

Capture usernames and passwords securely

Registration Forms

Collect details to create new accounts

Search Boxes

Let users search databases for results

3.1 Basic Form Structure

Every HTML form follows the same basic structure. The `<form>` tag wraps the inputs and defines **where** and **how** the data is sent. Inside it, you place input fields and a submit button to send the data.

Example

```
<form method="POST"
action="process.php">
  <input type="text" name="username">
  <input type="submit" value="Submit">
</form>
```

When the user clicks **Submit**, the browser sends the form data to `process.php` using the POST method.

Attribute Breakdown

- `form` - wraps all inputs
- `method` - sends data with GET or POST
- `action` - PHP file that processes the data
- `input` - where the user enters data
- `name` - key used to read the value in PHP

Understanding Form Attributes

Each HTML form attribute controls how data is collected and sent to PHP. Learn the core ones before writing server-side logic.

`form`

The outer container for all inputs. Without it, the form cannot submit data.

`method`

Sets the HTTP method: `GET` adds data to the URL, while `POST` sends it in the request body.

`action`

Sets the PHP file that receives and processes the data. If omitted, the form submits to itself.

`input`

Collects one piece of user data. Its `name` becomes the key used in PHP, such as `$_GET["username"]` or `$_POST["username"]`.

TOPIC 3 · PART 2

GET vs POST Methods

PHP forms send data with either **GET** or **POST**. The method you choose affects visibility, security, and what kind of data you can send.

GET

Appends data to the URL. It is visible, limited in size, and best for searches or read-only requests.

POST

Sends data in the request body. It stays out of the URL, has no practical size limit, and is best for sensitive or changing data.

GET Method

GET sends form data in the URL as a **query string**. It works well for search forms and filters, but never use it for passwords or private data.

Example

```
<form method="GET"
action="process.php">
  <input type="text" name="username">
  <input type="submit">
</form>
```

After submit, the URL becomes:

```
process.php?username=Kelvin
```

In PHP, retrieve it with: `$_GET["username"]`

Characteristics

- Data is visible in the browser address bar
- Limited to about 2,000 characters
- Can be bookmarked or shared by URL
- Less secure, so avoid passwords
- Best for search boxes, filters, and pagination

POST Method

The **POST** method sends data in the **HTTP request body**, so it stays hidden from the URL. It supports larger payloads and is the right choice for login forms, file uploads, and any action that creates or updates data.

Characteristics

- Data is **not** visible in the URL
- No practical size limit for form data
- Cannot be bookmarked or cached
- More secure for sensitive information
- Best for: login, registration, file uploads

Example

```
<form method="POST"
action="process.php">
  <input type="text" name="username">
  <input type="submit">
</form>
```

In PHP, retrieve the submitted value with:

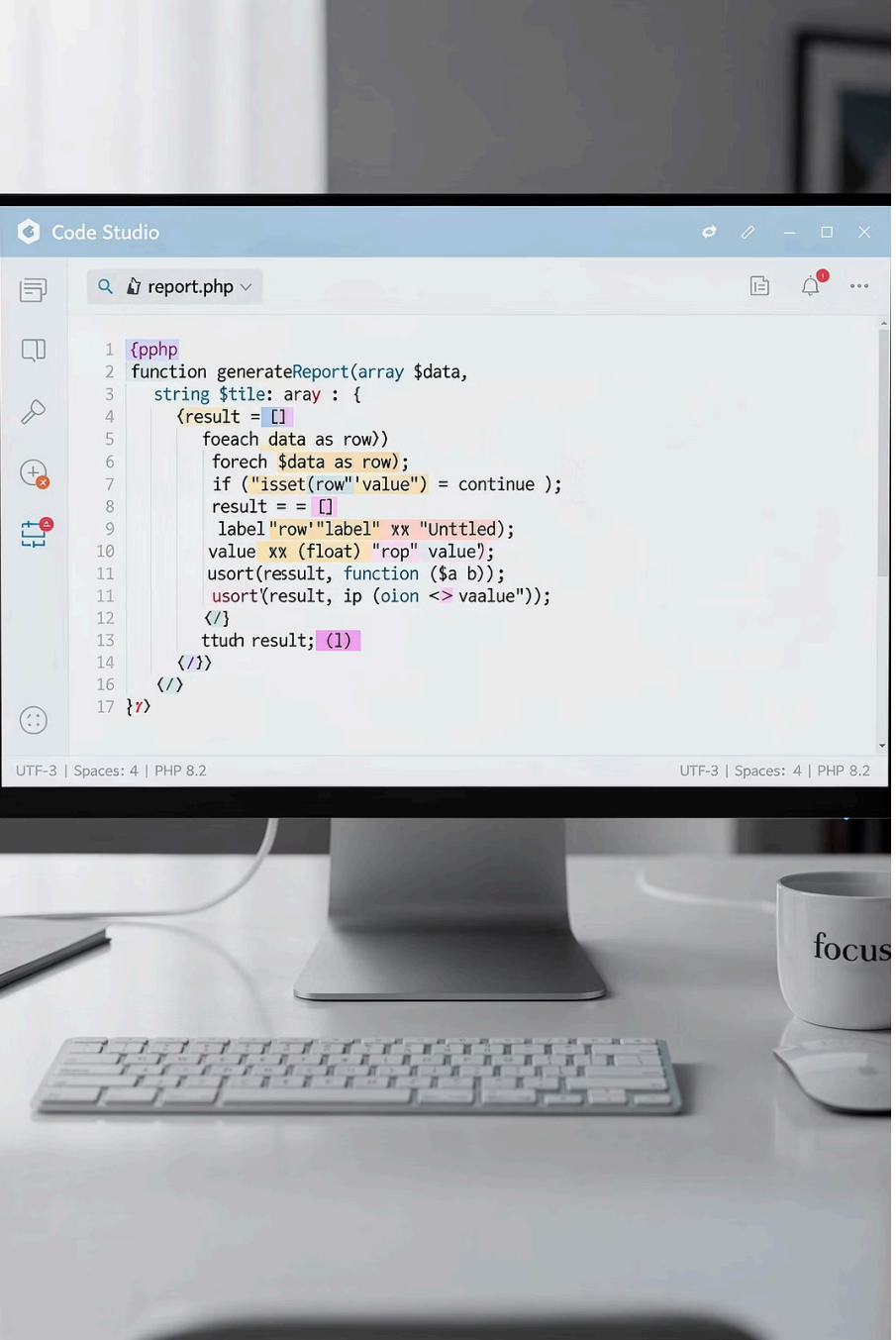
```
<?php
$username = $_POST["username"];
echo $username;
?>
```

GET vs POST - Key Differences

Use this quick comparison to choose the right method. Rule of thumb: **GET for reading, POST for writing**.

Feature	GET	POST
Data Visibility	Visible in URL	Hidden in request body
Security	Low - avoid sensitive data	Higher - use for passwords
Data Size Limit	~2,000 characters	No practical limit
Bookmarkable	Yes	No
PHP Superglobal	`$_GET["key"]`	`$_POST["key"]`
Best Used For	Search, filters, navigation	Login, registration, uploads

Part 3: Handling Form Data in PHP - Next, we'll write the PHP code that receives, validates, and responds to form submissions using `$_GET` and `$_POST`.



LEARNING OUTCOME 2.1.4

Functions in PHP

Reusable code

Use functions to write code once and reuse it.

4.1 Introduction to Functions

A function is a self-contained block of code that performs one specific task. It can be defined once and reused whenever needed.

4.1.1 Definition

A function is a self-contained block of code that performs a specific task.

- Define the logic once
- Reuse it throughout the program

4.1.2 Importance of Functions

Code Reusability

A function can be reused in different parts of a program without rewriting the same logic.

Improved Readability

Smaller functions make programs easier to understand.

Easier Maintenance

Update the function in one place instead of many.

Modularity

Programs can be split into smaller, manageable parts.

4.2 Defining and Calling Functions

Define a function with the function keyword, a name, and parentheses. To run it, call the function by its name.

4.2.1 Function Definition

Syntax:

```
function functionName() {
    // code to be executed
}
```

4.2.2 Function Call

Call the function by its name to execute it:

```
<?php
function greet() {
    echo "Welcome to the PHP class";
}

greet();
?>
```

Explanation

- function greet() defines the function
- The code inside {} runs when the function is called
- greet(); starts execution

PHP reads the full script before it runs the code.

4.3 Functions with Parameters

Parameters are variables passed into a function so it can work with different values.

4.3.1 Single Parameter

```php
<?php
function greet($name) {
    echo "Hello " . $name;
}

greet("Kelvin");
?>
```

Explanation

- $name is a parameter
- "Kelvin" is an argument passed to the function
- The function uses the parameter to produce output

4.3.2 Multiple Parameters

A function can accept more than one parameter:

```php
<?php
function add($a, $b) {
    echo $a + $b;
}

add(5, 3);
?>
```

This function takes two values and adds them.
Output: 8

4.4 Functions with Return Values

A function can return a value with the `return` statement. The result can then be stored or used elsewhere.

Example

```php
<?php
function multiply($x, $y) {
    return $x * $y;
}

$result = multiply(4, 5);
echo $result;
?>
```

Explanation

- The function multiplies the values
- `return` sends the result back
- The result is stored in `$result`
- Output: 20

Key point: once `return` runs, the function stops immediately.

4.5 Default Parameter Values

A parameter can have a default value. If no argument is passed, the function uses that value automatically.

Example

```php
<?php
function greet($name = "Guest") {
    echo "Hello " . $name;
}

greet();       // Output: Hello Guest
greet("Kelvin"); // Output: Hello Kelvin
?>
```

Explanation

- $name = "Guest" sets the default value
- If no value is passed, "Guest" is used
- A passed value overrides the default
- Useful for optional parameters

4.6 Types of Functions in PHP

4.6.1 Built-in Functions

PHP includes ready-made functions you can use right away.

```php
<?php
strlen("Hello");   // returns 5 - length of string
strtoupper("hi");  // returns HI
date("Y-m-d");     // returns current date
?>
```

4.6.2 User-defined Functions

These are custom functions you write to handle specific tasks.

```php
<?php
function displayMessage() {
    echo "This is a custom function";
}

displayMessage();
?>
```

PHP includes thousands of built-in functions for strings, arrays, math, dates, files, and more.

4.7 String Functions

String functions manipulate text data. They are among the most frequently used built-in functions in PHP.

Common Functions

`trim()`	Remove leading/trailing whitespace
`strlen()`	Get the length of a string
`strtoupper()`	Convert to uppercase
`strtolower()`	Convert to lowercase
`strpos()`	Find position of a substring
`str_replace()`	Replace text within a string
`substr()`	Extract part of a string

Example

```php
<?php
$text = " Hello PHP ";

echo trim($text);              // "Hello PHP"
echo strlen($text);            // 11
echo strtoupper($text);        // " HELLO PHP "
echo strpos($text, "PHP");     // 7
echo str_replace("PHP", "World", $text); // " Hello World "
echo substr($text, 1, 5);      // "Hello"
?>
```

Always use `trim()` when handling user input to remove accidental spaces before processing or storing data.

4.8 Date and Time Functions

Date and time functions handle timestamps and formatted dates. PHP uses Unix timestamps (seconds since Jan 1, 1970) internally.

Common Functions

`date()`	Format a date/time value
`time()`	Get current Unix timestamp
`strtotime()`	Convert a date string to timestamp
`mktime()`	Create a timestamp from specific values
`DateTime`	Class for advanced date handling

Example

```php
<?php
echo date("Y-m-d");           // e.g. 2025-04-15
echo date("d/m/Y H:i:s");     // e.g. 15/04/2025 10:30:00
echo time();                  // Current Unix timestamp
echo strtotime("next Monday"); // Timestamp of next Monday
echo date("Y-m-d", strtotime("+7 days")); // One week from now
?>
```

The `date()` format characters: Y = 4-digit year, m = month, d = day, H = hour, i = minutes, s = seconds.

4.9 Array Functions

Array functions manage, manipulate, and search arrays. They eliminate the need to write manual loops for common operations.

Common Functions

`count()`	Count number of elements
`array_push()`	Add element to end
`array_pop()`	Remove last element
`array_merge()`	Combine two or more arrays
`in_array()`	Check if value exists
`array_keys()`	Get all keys of an array
`sort()`	Sort array ascending
`array_reverse()`	Reverse array order

Example

```php
<?php
$arr = [1, 2, 3];

array_push($arr, 4);      // [1, 2, 3, 4]
array_pop($arr);          // [1, 2, 3]

echo count($arr);         // 3

$arr2 = [4, 5];
$merged = array_merge($arr, $arr2); // [1,2,3,4,5]

echo in_array(2, $arr);    // true (1)

print_r(array_keys($arr));
?>
```

Use `count()` instead of manually tracking array size — it always reflects the current number of elements.

4.10 Mathematical Functions

Math functions perform numeric operations — from rounding and absolute values to generating random numbers.

Common Functions

`abs()`	Absolute value (removes negative sign)
`round()`	Round to nearest integer
`ceil()`	Round up to next integer
`floor()`	Round down to next integer
`rand()`	Generate a random number
`max()`	Return the largest value
`min()`	Return the smallest value
`pow()`	Raise to a power
`sqrt()`	Square root

Example

```php
<?php
echo abs(-7);         // 7
echo round(4.5);      // 5
echo round(4.4);      // 4
echo ceil(4.1);       // 5
echo floor(4.9);      // 4
echo rand(1, 100);    // Random number between 1 and 100
echo max(3, 7, 2);    // 7
echo min(3, 7, 2);    // 2
echo pow(2, 3);       // 8
echo sqrt(16);        // 4
?>
```

Use ceil() for billing (always round up) and floor() for inventory (never count partial items).